

Unit 5

Searching And Indexing

Unit:5 Searching And Indexing

Searching is the process of locating specific information within a dataset by comparing a user's query against the available data. It involves techniques that identify matches, rank results, and return the most relevant information.

Indexing is the method of organizing data into structured forms such as lists, maps, or inverted indexes to make retrieval faster and more efficient. An index stores pointers to data so systems do not need to scan entire datasets.

Searching and indexing in Big Data refer to specialized techniques designed to efficiently retrieve relevant information from extremely large, diverse, and complex datasets. Traditional database search methods become slow and ineffective as data volume and variety increase, making advanced indexing approaches essential. Big Data platforms employ structures such as inverted indexes, tokenization, and distributed indexing to systematically organize data and minimize retrieval time. Searching then operates on these indexes by mapping user queries to stored terms, ensuring rapid, accurate, and scalable access to information across distributed environments. Together, these mechanisms form the foundation for high-performance information retrieval, real-time analytics, and large-scale search engine operations.

Full-Text Indexing and Searching

Full-text search refers to the process of searching entire textual documents to locate occurrences of specific words, phrases, or concepts. Unlike simple keyword matching, full-text search analyzes the content of documents at the word level, allowing efficient retrieval based on relevance, term frequency, and context. It enables fast querying over large collections of unstructured or semi-structured text, such as articles, logs, emails, or web pages.

Full-text search systems rely on specialized indexing structures that break down text into searchable components, making retrieval significantly faster than traditional sequential scanning.

Why Full-Text Search is Needed in Big Data?

In Big Data environments, datasets are vast, heterogeneous, and continuously growing. A large proportion of these datasets consist of unstructured text. Traditional database search mechanisms are not efficient for handling such volumes due to the following reasons:

- **High Volume and Velocity:** Big Data applications may require searching millions or billions of text documents in real time.
- **Unstructured Nature:** Logs, social media data, documents, and reports do not fit neatly into relational tables.
- **Complex Query Requirements:** Users often need features like phrase search, wildcard queries, fuzzy matching, and relevance ranking.
- **Performance Constraints:** Full table scans in relational databases become slow and impractical as data size increases.

Basic Concepts in Full-Text Indexing

a. Tokens

Tokens are the smallest meaningful units produced during the text-analysis phase. Tokenization divides a document into individual words or elements that can later be indexed. It may also manage punctuation, case differences, numbers, and special characters to ensure consistent processing.

Example:

Sentence: “Data indexing improves search efficiency.”

Tokens: {data, indexing, improves, search, efficiency}

b. Terms

A term is the standardized or normalized form of a token that is actually stored in the index. This normalization may involve converting text to lowercase, applying stemming or lemmatization, and unifying variations of words. Terms serve as the basic units used for matching queries to indexed content.

Example:

Tokens: “Running”, “runs”

Terms (after normalization): “run”

c. Stop Words

Stop words are frequently occurring words that carry minimal semantic value in search contexts. During indexing, these words are removed to reduce index size and improve retrieval efficiency by emphasizing content-bearing terms.

Examples: the, a, an, is, of, on, and

Basic Concepts in Full-Text Indexing

d. Stemming / Normalization

Stemming reduces words to their base or root form, while normalization applies broader standardization techniques such as lowercasing, removing punctuation, handling accents, or converting plurals to singular forms. These processes improve search accuracy by ensuring that related word forms are treated equivalently.

Example:

“connected”, “connection”, “connecting” → “connect”

e. Inverted Index

An inverted index is the fundamental data structure in full-text search systems. It maps each term to the documents in which it appears often including additional information such as frequency or exact positions. This structure supports extremely fast lookup because the search engine does not need to scan all documents.

Example:

Term	Document IDs
data	D1, D2, D5
search	D1, D3
indexing	D2, D4, D5

Full-Text Indexing and Search Process

1. Indexing Process

1. Document Collection

The system first gathers a collection of documents from various sources such as text files, log entries, emails, webpages, or database records. Each document is assigned a unique identifier (e.g., D1, D2, D3) to distinguish it during indexing and retrieval.

Example Documents:

D1: “The data indexing improves search efficiency.”

D2: “Search systems use indexing structures.”

2. Tokenization

Tokenization breaks each document into individual tokens (words or meaningful units). This is the first step that transforms raw text into analyzable components.

Example:

D1 → {The, Data, indexing, improves, search, efficiency}

D2 → {Search, systems, use, indexing, structures}

Indexing Process continued...

3. Normalization

Normalization converts tokens into a consistent format. This may involve:

- Converting all text to lowercase
- Removing punctuation
- Handling plural/singular forms
- Standardizing word variations

Example (after lowercasing):

D1 → {the, data, indexing, improves, search, efficiency}

D2 → {search, systems, use, indexing, structures}

4. Stop-Word Removal

Common functional words that do not add meaningful value to search (such as *the*, *is*, *of*, *and*) are removed to reduce index size and focus on content-bearing terms.

After Stop-Word Removal:

D1 → {data, indexing, improves, search, efficiency}

D2 → {search, systems, use, indexing, structures}

5. Stemming / Lemmatization

Stemming or lemmatization reduces terms to their base or root form, allowing the system to treat similar word variations equivalently.

Example:

improves → *improve*

structures → *structure*

After stemming:

D1 → {data, indexing, improve, search, efficiency}

D2 → {search, system, use, indexing, structure}

Indexing Process continued...

6. Inverted Index Construction

An inverted index is created by mapping each term to the list of documents in which it appears. For each term, the index stores:

- Document IDs
- Optionally: term frequency, positions, or weights

Example Inverted Index:

Term	Document IDs
data	D1
indexing	D1, D2
improve	D1
search	D1, D2
efficiency	D1
system	D2
use	D2
structure	D2

7. Index Storage

The final inverted index is compressed, optimized, and stored using search engine frameworks such as Lucene, Solr, or Elasticsearch. These systems maintain efficient data structures that support rapid query processing, ranking, and real-time updates.

Full-Text Indexing and Search Process

2. Searching Process

1. Query Parsing

When a user submits a query, the system first parses it by breaking it into tokens (words or meaningful units) and normalizing them. Normalization ensures consistency with the indexed terms and may include lowercasing, removing punctuation, and applying stemming or lemmatization.

Example:

User Query: “Best smartphones under 500 dollars”

Tokens after parsing and normalization: {best, smartphone, under, 500, dollar}

2. Term Lookup

Each normalized term from the query is looked up in the inverted index. The inverted index maps each term to the documents in which it appears, allowing the system to quickly identify candidate documents.

Example Inverted Index:

Term	Document IDs
best	D1, D2
smartphone	D1, D2, D3
under	D2, D3
500	D2, D3
dollar	D2, D3

Term Lookup Result:

best → D1, D2

smartphone → D1, D2, D3

under → D2, D3

500 → D2, D3

dollar → D2, D3

Searching Process continued...

3. Document Retrieval

The system identifies documents containing the query terms by merging or intersecting the document lists. Only documents containing the relevant terms are retrieved.

Example: Query terms: {best, smartphone, under, 500, dollar}

Matching documents:

D1: “Best smartphone deals this month” → contains best, smartphone

D2: “Top smartphones under 500 dollars in 2025” → contains all terms

D3: “Affordable smartphones priced around 500 dollars” → contains smartphone, under, 500, dollar

4. Ranking

Retrieved documents can be ranked based on relevance using various criteria:

- **Term Frequency (TF):** How often a term appears in a document
- **Inverse Document Frequency (IDF):** Rare terms are given higher weight
- **TF-IDF:** Combines TF and IDF for balanced scoring

5. Results Returned

Finally, the system presents the most relevant documents to the user in ranked order, ensuring the user sees the documents most likely to satisfy the query first.

Example Final Result (Ranked):

D2: “Top smartphones under 500 dollars in 2025”

D3: “Affordable smartphones priced around 500 dollars”

D1: “Best smartphone deals this month”

Differences Between Traditional Database Search and Full-Text Indexing

Aspect	Traditional Database Search (RDBMS)	Full-Text Indexing (Search Engines / Big Data Systems)
Search Method	Performs sequential scans of tables or text fields	Utilizes inverted indexes to directly map terms to documents
Performance	Slow for large datasets due to full table scans	Fast retrieval even for millions of documents using pre-built indexes
Text Analysis	Limited or absent; cannot tokenize, normalize, or stem words	Supports tokenization, normalization, stemming
Relevance Ranking	Not supported	Supported using scoring mechanisms such as TF, IDF, and TF-IDF
Fuzzy / Wildcard Search	Not supported	Supported, including phrase matching, proximity, and wildcard queries
Scalability	Poor; performance degrades significantly with large datasets	Highly scalable; optimized for distributed and massive text datasets
Index Type	Structured indexes for database fields	Inverted index optimized for unstructured or semi-structured text
Flexibility	Limited; cannot handle complex text queries efficiently	Flexible; accommodates natural language queries, Boolean operators, and advanced search features
Use Case	Structured data such as customer records, inventory tables	Unstructured or semi-structured text such as documents, logs, webpages, and social media
Big Data Suitability	Not suitable for large-scale text search	Well-suited for Big Data search and analytics

Indexing with Lucene

Apache Lucene is a high-performance, open-source information retrieval library written in Java. It provides the essential components for full-text search, including text analysis, indexing, and query processing. Lucene is not a complete search engine by itself; instead, it acts as a flexible toolkit that allows developers to integrate powerful search capabilities into applications. Built on core IR concepts such as tokenization and inverted indexing, Lucene forms the foundation of major search platforms like Apache Solr and Elasticsearch, which extend its features to deliver scalable, enterprise-level search solutions.

Key Features of Lucene

1. High-Performance Indexing and Searching:

Lucene is designed to deliver highly efficient text processing, enabling rapid document indexing and fast execution of complex search queries.

2. Advanced Text Analysis:

It provides a comprehensive text analysis framework that includes tokenizers, filters, stemmers, stop-word removal mechanisms, and a variety of language-specific analyzers to ensure accurate and meaningful text processing.

3. Inverted Index Structure:

Lucene employs an optimized inverted index, which facilitates quick identification and retrieval of documents containing specific terms, significantly improving search performance.

Indexing with Lucene

4. Flexible Document and Field Model:

It supports a versatile document structure, allowing developers to define multiple field types such as stored, indexed, and tokenized fields within each document for precise control over indexing behavior.

5. Powerful Query Capabilities:

Lucene offers a rich set of query types, including phrase queries, wildcard and fuzzy queries, Boolean operators, prefix matching, and advanced relevance-based scoring.

6. Customizable Scoring Algorithms:

The library incorporates sophisticated ranking models such as TF-IDF, and allows customization of scoring logic to meet specific retrieval requirements.

7. Scalability and Extensibility:

Lucene is capable of handling large-scale datasets consisting of millions of documents. Its modular architecture supports the creation of custom analyzers, query parsers, and index configurations, ensuring adaptability to diverse application needs.

Indexing Process in Lucene

Lucene employs a systematic procedure to transform raw textual data into an efficient and searchable index. The major steps involved in this process are as follows:

a. Document Creation

In the first step, raw textual data such as text files, articles, web pages, or log entries is converted into a Lucene *Document*. Each Document consists of multiple *Fields* representing attributes like title, content, author, or publication date.

Example:

Consider the following article we want to index:

- **Title:** “Introduction to Big Data”
- **Content:** “Big data refers to large and complex datasets.”

Lucene creates a Document that includes a *title* field and a *content* field, each representing different components of the article.

b. Field Definition

For each field in the Document, Lucene allows precise configuration of how that field should be indexed. A field can be indexed or non-indexed, stored or non-stored, and tokenized or non-tokenized. These configurations determine how the field contributes to searching and result retrieval.

Example:

For the above document:

- The **title** field may be stored, indexed, and tokenized so that it appears in search results and contributes to relevance scoring.
- The **content** field may be indexed and tokenized but not stored to save space while still allowing full-text search.

Indexing Process in Lucene

c. Text Analysis

Before indexing, Lucene processes tokenized fields using an Analyzer. This includes tokenization, converting text to lowercase, removing stop words, and applying stemming or filtering. This stage ensures the text is normalized and consistent for efficient indexing.

Example:

Given the content:

“Big data refers to large and complex datasets.”

A StandardAnalyzer may transform it into the following tokens:

- **big, data, refer, large, complex, dataset**

Here, stop words such as “to” and “and” are removed, and words like “datasets” are stemmed to “dataset.”

d. Building the Inverted Index

After analysis, Lucene constructs an inverted index. In this structure, each term is mapped to the documents in which it occurs, along with information about term frequency and term positions within the document.

Example:

From the analyzed tokens, the inverted index may look like:

Indexing Process in Lucene

Term	Document ID	Frequency	Positions
big	Doc1	1	0
data	Doc1	1	1
large	Doc1	1	3
dataset	Doc1	1	5

e. Segment Creation and Merging

Lucene writes index data into immutable segments. Initially, many small segments are created, especially during frequent indexing operations. Over time, Lucene automatically merges smaller segments into larger ones to improve search efficiency and reduce index fragmentation.

Example:

If indexing starts by creating **Segment A**, and subsequent indexing operations produce **Segment B** and **Segment C**, Lucene may later merge them into a single larger **Segment M**, resulting in faster query processing.

Searching Process in Lucene

a. Query Parsing

The search process begins when the user submits a query. Lucene parses the query and processes it using an Analyzer, applying rules similar to those used during the indexing phase. This ensures that the query terms are tokenized, normalized, lowercased, and stripped of stop words in the same manner as the indexed text.

Example:

If a user searches for:

- “Big Data Concepts”

The Analyzer may generate the tokens:

- **big, data, concept**

This ensures consistency between query processing and index processing.

b. Term Lookup

After parsing, Lucene searches the inverted index for documents that contain the processed query terms. Because the inverted index maps each term to its corresponding document IDs, Lucene can efficiently identify candidate documents without scanning the entire dataset.

Example:

From the inverted index, Lucene may find:

- Term **big** → appears in Doc1, Doc4
- Term **data** → appears in Doc1, Doc2, Doc4
- Term **concept** → appears in Doc3

Searching Process in Lucene

c. Scoring and Ranking

Lucene applies ranking algorithms, to compute a relevance score for each candidate document. The score is based on several factors:

- **Term Frequency (TF):** How often the query term appears in the document
- **Inverse Document Frequency (IDF):** How rare the term is across the entire index
- **Document Length Normalization:** Penalizing very long documents to avoid bias

Example:

Suppose the query is big data and Lucene identifies Doc1 and Doc4.

- If **Doc1** contains both “big” and “data” multiple times, it will have a higher TF.
- If “big” is a rare term in the index, IDF boosts its importance.
- If **Doc4** is much longer, its score may be reduced due to normalization.

As a result, **Doc1** may receive a higher relevance score than Doc4.

d. Retrieval of Matching Documents

Based on the computed scores, Lucene’s IndexSearcher retrieves and returns the documents in descending order of relevance. This ensures that the most relevant documents appear first in the search results.

Example:

Final ranked output for the query “**Big Data Concepts**” might be:

- **Doc1** – Highest relevance
- **Doc4** – Contains partial matches
- **Doc3** – Contains “concept” but not “big” or “data”

Advantages of Lucene

- **Fast Indexing and Searching Performance**
Lucene provides highly efficient mechanisms for text processing, enabling rapid document indexing and fast query execution even on large datasets.
- **Support for Complex Query Types and Relevance Ranking**
It offers a rich query language, allowing phrase queries, wildcard and fuzzy searches, Boolean operations, and relevance-based ranking for more accurate retrieval.
- **Highly Customizable Analyzers and Scoring Methods**
Lucene allows developers to customize analyzers, tokenizers, filters, and scoring algorithms, enabling fine-tuned search behavior suited to different application requirements.
- **Compact and Optimized Index Storage**
Lucene stores index data in a compact and optimized form, reducing storage overhead while maintaining high retrieval performance.
- **Scalability for Large Document Collections**
Its architecture is designed to scale efficiently, allowing the indexing and searching of millions of documents without significant performance degradation.
- **Foundation for Advanced Search Platforms**
Lucene serves as the core engine behind major enterprise search solutions such as Apache Solr and Elasticsearch, demonstrating its reliability, extensibility, and industry-wide adoption.

Use Cases of Lucene

- **Enterprise Search Systems**

Lucene is widely used to enable efficient searching of organizational content such as documents, reports, emails, and internal knowledge bases, improving information retrieval within enterprises.

- **Web and E-commerce Search Engines**

It powers product and category search, as well as recommendation systems, providing fast and relevant search results for online stores and web platforms.

- **Content Management Systems (CMS)**

Lucene is employed to index and search articles, blogs, research papers, and digital libraries, enhancing content accessibility and discoverability.

- **Log and Event Analysis**

Lucene facilitates searching and analyzing system logs or machine-generated data, helping in monitoring, troubleshooting, and real-time event detection.

- **Big Data Platforms**

Within distributed search and analytics platforms such as Apache Solr and Elasticsearch, Lucene provides the core indexing and search capabilities, enabling large-scale, high-performance data retrieval.

- **Code Search Tools**

Lucene is used to index and search large software codebases efficiently, supporting developers in quickly locating functions, classes, or specific code snippets.

Introduction to Apache Solr

Apache Solr is an open-source, enterprise-level search platform built on top of Apache Lucene. Unlike Lucene, which functions as a search library, Solr is a complete search server that provides a ready-to-use, highly scalable, and fault-tolerant solution for full-text search, and real-time indexing.

Solr extends the capabilities of Lucene by offering advanced features such as distributed search, caching, replication, and an HTTP-based RESTful API, which facilitates easy integration into web and enterprise applications. Its architecture is designed for high availability and performance, making it suitable for handling large-scale search requirements efficiently.

Solr is widely adopted in domains such as e-commerce, content management systems, big data analytics, and any application that demands fast, reliable, and complex search functionalities.

Key Features of Apache Solr

1. Built on Lucene

Solr leverages the powerful indexing and search capabilities of Apache Lucene, inheriting its efficient text processing and relevance ranking mechanisms.

2. Faceted Search

Solr provides faceted search, enabling users to filter and navigate search results based on categories, tags, or value ranges, improving result exploration and usability.

3. Distributed Search and Scalability

Through SolrCloud, Solr supports horizontal scaling across multiple servers, ensuring high availability, fault tolerance, and efficient handling of large data volumes.

Key Features of Apache Solr

4. **RESTful HTTP API**

Solr offers a standard RESTful HTTP interface, allowing seamless integration with web and enterprise applications for indexing, querying, and managing data.

5. **Real-Time Indexing**

Solr supports near real-time indexing, enabling newly added or updated documents to be searchable almost immediately.

6. **Advanced Querying**

Solr supports a wide range of query types, including Boolean queries, phrase searches, wildcard and fuzzy searches, range queries, and query boosting for prioritizing certain results.

7. **Rich Document Support**

Solr can index and search multiple document formats, including XML, JSON, CSV, and Microsoft Word/PDF files, making it suitable for diverse content types.

8. **Caching and Performance Optimization**

Solr incorporates query, filter, and document caches to enhance search performance and reduce response times.

9. **Extensibility**

The platform supports plugins for custom analyzers, query parsers, and request handlers, allowing developers to extend and customize Solr according to specific application needs.

Use Cases of Apache Solr

1. Enterprise Search

Solr is used to enable efficient search across large document repositories, intranet portals, and organizational knowledge bases.

Example: A company-wide search system allowing employees to quickly locate policy manuals, reports, and employee guides.

2. E-commerce Product Search

Solr supports fast product search with faceted navigation, filters, auto-suggestions, and relevance-based ranking.

Example: Online retail websites where users can filter products by price, brand, or category and receive instant search results.

3. Content Management Systems (CMS)

Solr indexes and searches blogs, articles, digital libraries, multimedia content, and news portals, improving content accessibility.

Example: News websites providing keyword and tag-based search across thousands of articles.

4. Big Data and Analytics Platforms

Solr integrates with Hadoop and other Big Data frameworks to index and query massive datasets efficiently.

Example: Analyzing large-scale user logs or transactional datasets for insights and reporting.

Use Cases of Apache Solr

5. Website Search Engines

Solr delivers fast and scalable search functionality for corporate or public-facing websites.

Example: University website search for courses, faculty profiles, events, and announcements.

6. Log and Event Analysis

Solr enables indexing and searching of server logs or system events to support monitoring, debugging, and alerting.

Example: IT teams using Solr to track error logs and monitor performance metrics across multiple servers.

7. Recommendation Systems

Solr can enhance personalized content or product recommendations using search and faceted navigation.

Example: Online stores suggesting products to users based on their previous searches or purchase history.

Distributed search with Elasticsearch

Elasticsearch (ES) is a distributed, scalable, and REST-driven search and analytics engine built on top of the Apache Lucene library. It is designed to efficiently handle large volumes of structured, semi-structured, and unstructured data while providing near real-time search performance. ES supports full-text search, powerful aggregations, and complex analytical queries across massive datasets. Its distributed architecture enables automatic data partitioning, replication, and load balancing, ensuring high availability and fault tolerance. With flexible schema support and compatibility with diverse data formats, Elasticsearch adapts well to various application needs.

Key Features of Elastic Search

1. Foundation on Apache Lucene

At its core, Elasticsearch relies on **Apache Lucene**, a high-performance full-text search library. While Lucene provides the underlying indexing and search algorithms, Elasticsearch extends it by offering a more user-friendly, distributed, and scalable interface. This abstraction allows developers to utilize Lucene's power without dealing directly with its low-level complexities.

2 REST-Based Interface

Elasticsearch communicates through a RESTful API, using simple HTTP methods such as GET, POST, PUT, DELETE, and PATCH. This design allows developers to interact with ES using any programming language capable of making HTTP requests. Queries and index operations are executed through JSON-based requests, making ES highly accessible and easy to integrate with external applications.

Distributed search with Elasticsearch

3 Distributed Architecture and Scalability

One of Elasticsearch's most defining characteristics is its **distributed architecture**. Data is automatically divided into **shards**, which can be stored across multiple nodes within a cluster. To ensure fault tolerance, Elasticsearch maintains **replica shards**, providing redundancy and enabling continuous availability even if a node fails.

This architecture ensures:

- **Horizontal scalability**
- **High availability**
- **Efficient load balancing across the cluster**

The distributed nature of Elasticsearch allows it to handle datasets ranging from small collections to massive, multi-terabyte indices used in enterprise environments.

4 Full-Text Search Capabilities

Elasticsearch is optimized for **full-text search**, utilizing techniques such as:

- **Inverted indexing**
- **Tokenization and text analysis**
- **Relevance scoring (TF-IDF and BM25)**
- **Fuzzy search and wildcard search**
- **Phrase queries**

These capabilities enable ES to return highly relevant results quickly, even from large textual datasets. Its search engine features are widely used in applications such as e-commerce product search, document retrieval systems, and knowledge bases.

Distributed search with Elasticsearch

5 Real-Time Data Indexing

A key advantage of Elasticsearch is its near real-time indexing. When documents are inserted or updated, they become searchable within a short latency window (typically within a second). This makes ES ideal for log and event processing, where data must be searchable almost immediately after being generated.

Use Cases and Applications of Elasticsearch

Elasticsearch is designed to support a broad range of real-world applications, including:

- **Log and Event Analysis:** When combined with Logstash and Kibana (forming the ELK/Elastic Stack), ES becomes a powerful log analytics platform for system monitoring and security analysis.
- **Enterprise Search:** Indexing documents, files, and internal data sources for rapid retrieval.
- **E-commerce Search Engines:** Facilitating product search with filters, facets, auto-suggestions, and relevance ranking.
- **Time-Series Analytics:** Storing and analyzing time-stamped data such as metrics, application activity, or IoT device readings.
- **Content Personalization and Recommendation Systems:** Enabling fast retrieval and ranking of personalized results.

Distributed search with Elasticsearch

Key Concepts in Elasticsearch

- **Cluster:** A cluster is a collection of one or more Elasticsearch nodes that work together to store data, distribute workloads, and provide search and analytics capabilities. It ensures scalability and high availability.
- **Node:** A node is a single running Elasticsearch instance (a Java process) that is part of a cluster. Each node can store data, handle queries, and perform different roles depending on its configuration.
- **Index:** An index is a logical grouping of documents, similar to a database in relational systems. It defines how data is organized and provides the main structure through which documents are stored and searched.
- **Document:** A document is the basic unit of information stored in Elasticsearch. It is represented in JSON format and is comparable to a row in a relational database, but can contain nested or complex data structures.
- **Shard:** A shard is a horizontal division of an index that allows data to be split and stored across multiple nodes. Sharding enables Elasticsearch to scale efficiently and handle large datasets by supporting parallel processing.
- **Replica:** A replica is a copy of a primary shard. Replicas increase fault tolerance ensuring data remains available if a node fails and improve read performance by allowing queries to be served from multiple copies.

Distributed search with Elasticsearch

How Distributed Search Works in Elasticsearch

Step 1: Client Sends a Search Request

When a client issues a search query, the request can be received by any node in the Elasticsearch cluster. This receiving node becomes the coordinating node.

Role of the Coordinating Node

- Interprets the client's search request
- Determines which shards need to be queried
- Forwards the query to the appropriate shards
- Collects and merges results
- Sends the final response back to the client

Example

Suppose an index named **products** is divided into 3 primary shards:

- Shard 0 (Node A)
- Shard 1 (Node B)
- Shard 2 (Node C)

If a client searches for "**smartphone**", and sends the request to Node B, then:

Node B becomes the coordinating node

It will forward the search query to Shard 0, Shard 1, and Shard 2

How Distributed Search Works in Elasticsearch

Step 2: Query Phase (Scatter Phase)

This is known as the scatter phase because the coordinating node distributes the search query to all relevant shards.

What Happens in Each Shard

Each shard executes the search locally, performing:

- Text analysis
- Relevance scoring
- Filtering
- Ranking
- Computing top document hits

Each shard returns only its top results rather than the entire dataset, which improves efficiency.

The Shard Response Includes:

- Matching documents
- Relevance scores
- Metadata like document IDs and shard information

Example

In the **products** index:

- Shard 0 finds 10 matching smartphones
- Shard 1 finds 5
- Shard 2 finds 12

Each shard might send back its top 5 ranked results to the coordinating node.

How Distributed Search Works in Elasticsearch

Step 3: Reduce Phase (Gather Phase)

In this phase, the coordinating node gathers all partial results returned from each shard.

What the Coordinating Node Does

- Merges results from all shards
- Re-calculates global ranking
- Sorts documents by relevance score, date, or any specified field
- Applies pagination (from/size)
- Generates the final unified result list

Finally, the node sends the final response back to the client.

Example

If each shard returned 5 top results (15 total), the coordinating node:

- Sorts all 15 documents together
- Selects the top 10 or the number the client requested
- Sends the final ranked list of products (e.g., smartphones) to the user

Data store, list, update index and delete with elasticsearch.

Managing data in Elasticsearch involves several key operations: storing, listing, updating, and deleting documents in an index. Here's a brief overview of each operation:

1. Data Store (Indexing)

To store a document, you use the PUT or POST request. The document is indexed in a specified index.

Example: Index a Document

```
PUT /my_index/_doc/1
{
  "title": "Elasticsearch Basics",
  "author": "John Doe",
  "content": "This is a guide to Elasticsearch."
}
```

2. List Documents (Searching)

To retrieve documents from an index, can use the GET request with a search query.

Example: Search Documents

```
GET /my_index/_search
{
  "query": {
    "match": {
      "author": "John Doe"
    }
  }
}
```

Data store, list, update index and delete with elasticsearch.

3. Update Document

To update an existing document, you can use the POST request with the `_update` endpoint.

Example: Update a Document

```
POST /my_index/_doc/1/_update
{
  "doc": {
    "content": "This is an updated guide to Elasticsearch."
  }
}
```

4. Delete Document

To delete a document, you use the DELETE request.

Example: Delete a Document

```
DELETE /my_index/_doc/1
```

Thank You!